

Security Assessment: Applicability and Scope Statement

Covers requested items 3, 4 and 5

Product: KRC Pune — VR / Amenities Sales Experience Application **Vendor:** Fute Services **Document date:** 16 July 2026 **Assessed at commit:** `5876920`, branch `main`

Please read this first

This document is a vendor self-declaration of scope, together with an internal secure-code review carried out by inspecting the repository.

It is not a VAPT report. No penetration test, dynamic testing or third-party security assessment has been performed on this product, and this document must not be represented as one.

Items 3, 4 and 5 are answered as not applicable because the assets they refer to do not exist in this product. That is a different statement from saying testing was performed and passed, and we want the distinction to be explicit.

1. Scope of the product

Asset type	Present	Consequence
Windows desktop application (Electron)	Yes	This is the entire product
Mobile application (Android or iOS)	No	Items 3 and 4 are not applicable
Backend API or server	No	Item 5 is not applicable
Database	No	No data-tier assessment applies
Web application or portal	No	—
User accounts or authentication	No	No authentication or authorisation attack surface

2. Items 3 and 4: mobile app VAPT and mobile source code review

Not applicable, because no mobile application exists.

KRC Pune is delivered exclusively as a Windows desktop application through an NSIS installer. There is no Android application, no iOS application and no mobile build target.

The evidence is straightforward. `code/package.json` declares a single build target, `"win": { "target": "nsis" }`, and contains no Android, iOS, Capacitor, Cordova or React Native configuration. No mobile framework appears anywhere in the dependency tree. The CI pipeline in `.github/workflows/build-windows.yml` produces exactly one artifact, a Windows installer.

One point that could cause confusion on review: some interface code contains responsive breakpoints named base, md and lg. That is layout code for varying desktop window sizes. It is not evidence of a mobile distribution.

If the underlying requirement is a security assessment of the artefact we actually deliver, then the in-scope asset is the Windows desktop application. We can commission a desktop-application VAPT if KRC requires one; a suggested scope is in section 5.

3. Item 5: API VAPT report

Not applicable, because the product ships no API and makes no API calls at runtime.

Every data-consuming screen in the shipped application reads from `src/data/offlineApi.ts`, which returns statically imported local JSON. The former backend was removed when the product moved to an offline build.

This was verified against the compiled production bundle, not only the source. Scanning `code/dist/assets/`, which is the code that actually ships inside the installer, returns no external service endpoints. The only matches are documentation URLs inside third-party library comments and error messages, such as react.dev, reactrouter.com, mui.com and w3.org.

Screen	Data source in the shipped build
VR tour (<code>pages/VR/VRPage.tsx</code>)	<code>getVrTour()</code> , local <code>vrTour.json</code>
Gallery (<code>pages/Overview/GalleryPage.tsx</code>)	<code>getGallery()</code> , local <code>gallery.json</code>
Vertical transport	<code>getTransport()</code> , local <code>transport.json</code>
Mobility	Local JSON; no network import present
Floor plans and units	<code>getFloors()</code> , local <code>floors.json</code>
Location	Bundled local MP4 with static image markers

3.1 Disclosure: dormant backend code remains in the repository

We are raising this ourselves rather than leaving your reviewers to find it and reasonably question the claim above.

The repository still contains unused code that references a live backend.

File	Contains	Ships
<code>src/api/axiosInstance.ts</code>	<code>baseUrl: "https://api.krpune1.futeservices.in/api"</code> , note the leading space	No; not imported by any routed page
<code>src/reusableCustome/api.ts</code>	<code>BASE_URL = "https://api.krpune1.futeservices.in/api"</code>	No; imported nowhere
<code>src/services/floorServices.ts</code>	<code>axiosInstance.get("/floors")</code>	No
<code>src/components/Overview/GalleryPage.tsx</code> , <code>src/components/ProjectDetails/MobilityPage.tsx</code> , <code>src/components/ProjectDetails/VerticalTransportPage.tsx</code> and other orphaned duplicate pages	<code>axios.get(...)</code> against <code>krahejabackend.onrender.com</code> and <code>http://103.133.214.185:5001</code>	No; <code>src/app/router.tsx</code> routes only <code>src/pages/**</code>

Three independent facts confirm this code is genuinely inert in the shipped build. A static import-reachability trace from the entry point `src/main.tsx` shows that only 46 of 96 code files under `src/` are reachable, and every module listed above is unreachable, so Vite never bundles it. The `axios` package is not even declared as a dependency in `code/package.json`, so any attempt to actually use this code would fail the build. And empirically, none of these hosts appears in the compiled bundle.

The runtime risk is low, since dead code cannot execute. The real risk is to correctness and to the audit trail. A future developer could import one of these modules and silently reintroduce a live network dependency, which would invalidate every claim in this pack. It also creates the appearance of an undisclosed backend, which is a poor position to be in during a client security review.

We recommend removing the network-referencing dead modules:

```
code/src/api/axiosInstance.ts
code/src/reusableCustome/api.ts
code/src/reusableCustome/useFetch.tsx
code/src/services/floorServices.ts
code/src/Route/route.tsx      (superseded by src/app/router.tsx)
```

One important caution for whoever does this work. Do not delete `src/components/` wholesale. The directory is a mix of live and dead code, and these three files are reachable and required, so removing them breaks the build:

- `src/components/Sidebar/Sidebar.tsx` , used by `src/app/RootLayout.tsx`
- `src/components/DayNightToggle/DayNightToggle.tsx` , used by `pages/Home/index.tsx`
- `src/components/ButtonDiv.tsx` , used by `pages/ProjectDetails/AboutUs.tsx`

Note also the near-identical dead file `src/components/ButtenDiv.tsx` , which is misspelt; the correctly spelt `ButtonDiv.tsx` is the live one. The duplicate page components under `src/components/Overview/` , `src/components/ProjectDetails/` and `src/components/Amenities/` should only be removed after each is checked against the reachability list, and the build re-run to verify.

This cleanup is optional as far as KRC's request is concerned. It does not change the security posture of the delivered installer, because none of this code ships. We recommend it so that the repository matches the claims made in this pack.

A separate question needs an answer internally: if `api.krpune1.futeservices.in` is live, it is Fute Services infrastructure that is out of scope for this product but may still be internet-exposed. We should confirm whether it is decommissioned. If it is still serving, it should either be taken down or disclosed to KRC separately. It should not be left undeclared while we claim the product has no API.

4. Internal secure-code review of the desktop application

The findings below come from source inspection. To repeat the caveat at the top of this document, this is a self-review and not a VAPT.

4.1 Controls already in place

Control	Status	Location
<code>contextIsolation</code>	Enabled	<code>electron/main.cjs</code>
<code>nodeIntegration</code>	Disabled	<code>electron/main.cjs</code>
Preload IPC bridge	Deliberate no-op; no privileged surface exposed	<code>electron/preload.cjs</code>
External URL containment	Window-open handler denies in-app loads; navigation to non- <code>file://</code> URLs is blocked	<code>electron/main.cjs</code>
Remote content loading	None; <code>loadFile()</code> over <code>file://</code> only	<code>electron/main.cjs</code>
Secrets in version control	Partial; <code>.env</code> is not tracked, but see finding 7	<code>code/.gitignore</code>
Attack surface	No authentication, PII, storage, forms, telemetry or listening port	Verified by search

4.2 Findings requiring action

#	Severity	Finding	Recommendation
1	Medium	The installer is not code-signed, and users are instructed to bypass the SmartScreen warning. The integrity and authenticity of a roughly 1 GB executable cannot be verified	Obtain an OV or EV code-signing certificate. CI already supports it: set the <code>WINDOWS_CERT_BASE64</code> and <code>WINDOWS_CERT_PASSWORD</code> repository secrets and builds sign automatically, with no code change
2	Medium	Public distribution. The installer is served from a public GitHub Release with a stable link, so bundled marketing media and floor plans are downloadable by anyone holding it	Move to an authenticated or private channel, or hand-deliver, if KRC considers the content sensitive
3	Low	Dormant backend code referencing three different API hosts, as set out in section 3.1	Delete the dead modules listed there, observing the caution about <code>src/components/</code>
4	Low	Unused secret in the local <code>.env</code> : <code>VITE_MAPBOX_TOKEN</code> . Nothing in the codebase reads it, since <code>MapboxMap.tsx</code> uses a bundled MP4 rather than the Mapbox SDK. Note that any <code>VITE_</code> -prefixed variable is inlined into the client bundle at build time and is public by design	Delete the variable and revoke or rotate the token at Mapbox. It has been distributed across developer environments and serves no purpose
5	Low	No update or patch channel. A vulnerability in a bundled dependency such as Electron or Three.js requires manual redistribution	Agree a patch-notification and redistribution process with KRC. Track Electron security releases
6	Informational	Electron 33 is not the latest major release. No exploit path exists here given that no remote content is loaded, but the bundled Chromium ages	Plan a periodic Electron upgrade cadence
7	Medium	The repository root has no <code>.gitignore</code> ; <code>code/.gitignore</code> only governs <code>code/</code> . The root <code>.env</code> , which holds <code>VITE_MAPBOX_TOKEN</code> , is untracked but not ignored, so a single <code>git add -A</code> would commit it to a repository with public releases. <code>node_modules/</code> , <code>dist/</code> at roughly 644 MB, and <code>release/</code> are likewise unignored at root	Add a root <code>.gitignore</code> covering <code>.env*</code> , <code>node_modules/</code> , <code>dist/</code> and <code>release/</code> . This has been fixed; see the repository root. Rotate the Mapbox token regardless, per finding 4

All findings above are open as of commit `5876920`, with the exception of finding 7, which is fixed. Findings 3 and 4 should be closed before this pack is sent to KRC, because both directly undercut the offline, no-API statement we are making.

5. If KRC requires a formal VAPT regardless

Should your policy mandate a VAPT despite the not-applicable determinations above, the only in-scope asset is the Windows desktop application. A meaningful scope would cover an Electron application security review against the OWASP and Electron hardening checklists; installer and binary integrity analysis together with code-signing verification; local privilege and file-permission analysis of the installed footprint; a dependency vulnerability scan of the bundled third-party components; and independent runtime network monitoring on a live installation to verify the no-outbound-traffic claim rather than take our word for it.

This would need to be commissioned from a qualified third-party assessor. Fute Services cannot self-issue a VAPT report.

Suggested wording for the reply to KRC IT:

Items 3 to 5 relate to mobile and API assets which this product does not contain; it is a fully offline Windows desktop application with no backend. We have supplied network and data-flow diagrams evidencing this, together with an internal secure-code review. If your policy requires independent testing of the delivered artefact, we propose a desktop-application VAPT with the scope above and would welcome your confirmation.

6. Attestation

Fute Services declares that, at commit `5876920`, KRC Pune contains no mobile application and no API, and performs no network communication at runtime. This declaration is based on source-code and compiled-bundle inspection. It is not the result of

independent penetration testing.

Prepared by *[Name, Title]*, Fute Services, 16 July 2026

Reviewed and approved by *[Name, Title]*, *[Date]*